

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №6

з дисципліни «Бази даних»

Виконав:

Степаненко Денис
студент групи ІМ-051
номер у списку групи: 16

Перевірів:
Русінов В.В.

Київ 2026

Короткий виклад вимог

Завдання:

Використати Prisma ORM для роботи з базою даних VideoHub. Описати схему в `schema.prisma`, створити міграції для зміни схеми, перевірити результат через Prisma Studio.

Що таке Prisma ORM:

Prisma — Object-Relational Mapping (ORM) інструмент для Node.js/TypeScript. Замість написання SQL-запитів вручну, Prisma генерує типобезпечний клієнт з `schema.prisma`. Міграції автоматично створюють SQL для зміни структури БД.

Що таке міграція:

Міграція — контрольована зміна структури БД. Prisma порівнює `schema.prisma` з поточним станом БД і генерує SQL-файл з потрібними змінами (CREATE TABLE, ALTER TABLE, DROP). Кожна міграція зберігається в `prisma/migrations/` — це історія змін схеми.

Поняття:

- **schema.prisma** — декларативний опис моделей БД (аналог SQL CREATE TABLE, але зручніший)
- **prisma migrate dev** — команда для створення та застосування міграції
- **prisma generate** — генерація TypeScript-клієнта з `schema.prisma`
- **prisma studio** — GUI для перегляду та редагування даних
- **@relation** — опис зв'язку між моделями (FK)
- **@map** — мапінг імені поля в БД (наприклад, `authorId` → `author_id`)

Ініціалізація Prisma

Створюємо Node.js-проєкт з Prisma:

```
{
  "name": "videohub-prisma",
  "version": "1.0.0",
  "devDependencies": {
    "prisma": "^6.0.0"
  },
}
```

```
"dependencies": {
  "@prisma/client": "^6.0.0"
}
```

Підключення до PostgreSQL через `.env` :

```
DATABASE_URL="postgresql://postgres:5643@localhost:5432/videohub?schema=public"
```

schema.prisma — початкова схема

Описуємо моделі VideoHub (User, Video, Comment, Like, Subscription):

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String  @id @default(uuid()) @db.Uuid
  email       String  @unique
  username    String  @unique
  password    String?
  googleId    String? @unique
  description String?
  avatar      String?
  banner      String?
  createdAt   DateTime @default(now()) @map("created_at")

  videos      Video[]
  comments    Comment[]
  likes       Like[]
  subscribers Subscription[] @relation("SubscriberRelation")
  subscriptions Subscription[] @relation("SubscribedToRelation")

  @@map("User")
}

model Video {
  id          String  @id @default(uuid()) @db.Uuid
  title       String
  description String?
  url         String
  thumbnail   String?
  views       Int      @default(0)
  createdAt   DateTime @default(now()) @map("created_at")
}
```

```

authorId    String    @map("author_id") @db.Uuid

author      User      @relation(fields: [authorId], references: [id], onDelete: Cascade)
comments    Comment[]
likes       Like[]

@@map("Video")
}

model Comment {
  id         String    @id @default(uuid()) @db.Uuid
  text       String
  createdAt  DateTime  @default(now()) @map("created_at")
  userId     String    @map("user_id") @db.Uuid
  videoId    String    @map("video_id") @db.Uuid

  user       User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  video      Video     @relation(fields: [videoId], references: [id], onDelete: Cascade)

  @@map("Comment")
}

model Like {
  id         String    @id @default(uuid()) @db.Uuid
  createdAt  DateTime  @default(now()) @map("created_at")
  userId     String    @map("user_id") @db.Uuid
  videoId    String    @map("video_id") @db.Uuid

  user       User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  video      Video     @relation(fields: [videoId], references: [id], onDelete: Cascade)

  @@unique([userId, videoId])

  @@map("Like")
}

model Subscription {
  id         String    @id @default(uuid()) @db.Uuid
  createdAt  DateTime  @default(now()) @map("created_at")
  subscriberId String  @map("subscriber_id") @db.Uuid
  subscribedToId String @map("subscribed_to_id") @db.Uuid

  subscriber User      @relation("SubscriberRelation", fields: [subscriberId],
references: [id], onDelete: Cascade)
  subscribedTo User     @relation("SubscribedToRelation", fields: [subscribedToId],
references: [id], onDelete: Cascade)

  @@unique([subscriberId, subscribedToId])

  @@map("Subscription")
}

```

Особливості `schema.prisma`:

- `@id @default(uuid())` — первинний ключ, авто-генерація UUID
- `@db.Uuid` — тип колонки в PostgreSQL (UUID)
- `@unique` — унікальне обмеження
- `@map("column_name")` — мапінг імені поля (camelCase в кодї → snake_case в БД)
- `@@map("table_name")` — мапінг імені таблиці
- `@@unique([field1, field2])` — складене унікальне обмеження
- `@relation(fields: [fk], references: [pk], onDelete: Cascade)` — зв'язок FK
- `String?` — nullable поле (знак питання)

Міграція 1: init

Початкова схема — створення всіх таблиць VideoHub:

```
npx prisma migrate dev --name init
```

Prisma генерує SQL-файл `migration.sql` :

```
CREATE TABLE "User" (  
  "id" UUID NOT NULL DEFAULT gen_random_uuid(),  
  "email" VARCHAR(255) NOT NULL,  
  "username" VARCHAR(100) NOT NULL,  
  "password" VARCHAR(255),  
  "google_id" VARCHAR(255),  
  "description" TEXT,  
  "avatar" VARCHAR(500),  
  "banner" VARCHAR(500),  
  "created_at" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT "User_pkey" PRIMARY KEY ("id")  
);  
  
CREATE TABLE "Video" (  
  "id" UUID NOT NULL DEFAULT gen_random_uuid(),  
  "title" VARCHAR(255) NOT NULL,  
  "description" TEXT,  
  "url" VARCHAR(500) NOT NULL,  
  "thumbnail" VARCHAR(500),  
  "views" INTEGER NOT NULL DEFAULT 0,  
  "created_at" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  "author_id" UUID NOT NULL,  
  CONSTRAINT "Video_pkey" PRIMARY KEY ("id")  
);  
  
-- ... Comment, Like, Subscription аналогічно  
  
-- Унікальні індекси  
CREATE UNIQUE INDEX "User_email_key" ON "User"("email");  
CREATE UNIQUE INDEX "User_username_key" ON "User"("username");  
CREATE UNIQUE INDEX "Like_user_id_video_id_key" ON "Like"("user_id", "video_id");
```

```
-- Зовнішні ключі
ALTER TABLE "Video" ADD CONSTRAINT "Video_author_id_fkey"
    FOREIGN KEY ("author_id") REFERENCES "User"("id") ON DELETE CASCADE;
```

Міграція 2: add-video-duration

Додаємо поле `duration` (тривалість відео в секундах) до моделі `Video`:

```
model Video {
  // ... існуючі поля
  duration    Int?    @map("duration")    // нове поле
}
```

```
npx prisma migrate dev --name add_video_duration
```

Згенерований SQL:

```
ALTER TABLE "Video" ADD COLUMN "duration" INTEGER;
```

`Int?` — nullable (не всі відео мають відому тривалість при завантаженні). `ALTER TABLE ADD COLUMN` додає колонку до існуючої таблиці без втрати даних.

Міграція 3: add-video-category

Додаємо таблицю `Category` та зв'язок `Video` → `Category` (M:1):

```
model Category {
  id    Int    @id @default(autoincrement())
  name  String @unique

  videos Video[]

  @@map("Category")
}

model Video {
  // ... існуючі поля
  category    Category? @relation(fields: [categoryId], references: [id])
  categoryId  Int?      @map("category_id")    // нове поле FK
}
```

```
npx prisma migrate dev --name add_video_category
```

Згенерований SQL:

```
CREATE TABLE "Category" (  
  "id" SERIAL NOT NULL,  
  "name" VARCHAR(100) NOT NULL,  
  CONSTRAINT "Category_pkey" PRIMARY KEY ("id")  
);  
  
CREATE UNIQUE INDEX "Category_name_key" ON "Category"("name");  
  
ALTER TABLE "Video" ADD COLUMN "category_id" INTEGER;  
  
ALTER TABLE "Video" ADD CONSTRAINT "Video_category_id_fkey"  
  FOREIGN KEY ("category_id") REFERENCES "Category"("id")  
  ON DELETE SET NULL ON UPDATE CASCADE;
```

Category? — nullable FK (відео може не мати категорії). ON DELETE SET NULL — при видаленні категорії, category_id відео стає NULL (не видаляється відео). @default(autoincrement()) — автоінкремент для id категорії (SERIAL в PostgreSQL).

Prisma Studio

Перевірка даних через Prisma Studio — GUI для перегляду та редагування:

```
npx prisma studio
```

Prisma Studio відкривається в браузері на порту 5555. Показує всі таблиці з `schema.prisma`, дозволяє переглядати, додавати, редагувати та видаляти записи.

Скріншоти

schema.prisma — опис моделей VideoHub (частина 1: generator, datasource, User)

```

1 // =====
2 // Lab 6: Prisma ORM – schema.prisma
3 // Проект: VideoHub (відеохостинг-платформа)
4 // Степаненко Денис, ІМ-051
5 // =====
6
7 generator client {
8   provider = "prisma-client-js"
9 }
10
11 datasource db {
12   provider = "postgresql"
13   url      = env("DATABASE_URL")
14 }
15
16 // =====
17 // МОДЕЛІ (відповідають таблицям з Lab 2)
18 // =====
19
20 model User {
21   id          String   @id @default(uuid()) @db.Uuid
22   email       String   @unique
23   username    String   @unique
24   password    String?
25   googleId    String?  @unique
26   description String?
27   avatar      String?
28   banner      String?
29   createdAt   DateTime @default(now()) @map("created_at")
30
31   videos      Video[]
32   comments    Comment[]
33   likes       Like[]
34   subscribers Subscription[] @relation("SubscriberRelation")
35   subscriptions Subscription[] @relation("SubscribedToRelation")
36
37   @@map("User")
38 }
39
40 model Video {
41   id          String   @id @default(uuid()) @db.Uuid
42   title       String
43   description String?
44   url         String
45   thumbnail   String?
46   views       Int       @default(0)
47   duration    Int?      @map("duration")
48   createdAt   DateTime @default(now()) @map("created_at")

```

schema.prisma — моделі Video, Comment, Like (частина 2)

```

47   duration    Int?    @map("duration")
48   createdAt   DateTime @default(now()) @map("created_at")
49   authorId    String   @map("author_id") @db.Uuid
50
51   author      User     @relation(fields: [authorId], references: [id], onDelete: Cascade)
52   comments    Comment[]
53   likes       Like[]
54   category    Category? @relation(fields: [categoryId], references: [id])
55   categoryId  Int?     @map("category_id")
56
57   @@map("Video")
58 }
59
60 model Comment {
61   id          String   @id @default(uuid()) @db.Uuid
62   text        String
63   createdAt   DateTime @default(now()) @map("created_at")
64   userId      String   @map("user_id") @db.Uuid
65   videoId     String   @map("video_id") @db.Uuid
66
67   user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
68   video       Video    @relation(fields: [videoId], references: [id], onDelete: Cascade)
69
70   @@map("Comment")
71 }
72
73 model Like {
74   id          String   @id @default(uuid()) @db.Uuid
75   createdAt   DateTime @default(now()) @map("created_at")
76   userId      String   @map("user_id") @db.Uuid
77   videoId     String   @map("video_id") @db.Uuid
78
79   user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
80   video       Video    @relation(fields: [videoId], references: [id], onDelete: Cascade)
81
82   @@unique([userId, videoId])
83
84   @@map("Like")
85 }
86
87 model Subscription {
88   id          String   @id @default(uuid()) @db.Uuid
89   createdAt   DateTime @default(now()) @map("created_at")
90   subscriberId String   @map("subscriber_id") @db.Uuid
91   subscribedToId String @map("subscribed_to_id") @db.Uuid
92

```

schema.prisma — моделі Subscription, Category (частина 3)

```

90     subscriberId String @map("subscriber_id") @db.Uuid
91     subscribedToId String @map("subscribed_to_id") @db.Uuid
92
93     subscriber User @relation("SubscriberRelation", fields: [subscriberId], references: [id], onE
94     subscribedTo User @relation("SubscribedToRelation", fields: [subscribedToId], references: [id],
95
96     @@unique([subscriberId, subscribedToId])
97
98     @@map("Subscription")
99 }
100
101 // =====
102 // МОДЕЛІ ДОДАНІ ЧЕРЕЗ МІГРАЦІЇ
103 // =====
104
105 model Category {
106     id Int @id @default(autoincrement())
107     name String @unique
108
109     videos Video[]
110
111     @@map("Category")
112 }
113

```

Міграція 1 — init: CREATE TABLE (частина 1)

```

MyWork > Lab_6 > prisma > migrations > 20260916000000_init > migration.sql
1  -- CreateTable
2  CREATE TABLE "User" (
3      "id" UUID NOT NULL DEFAULT gen_random_uuid(),
4      "email" VARCHAR(255) NOT NULL,
5      "username" VARCHAR(100) NOT NULL,
6      "password" VARCHAR(255),
7      "google_id" VARCHAR(255),
8      "description" TEXT,
9      "avatar" VARCHAR(500),
10     "banner" VARCHAR(500),
11     "created_at" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
12
13     CONSTRAINT "User_pkey" PRIMARY KEY ("id")
14 );
15
16 -- CreateTable
17 CREATE TABLE "Video" (
18     "id" UUID NOT NULL DEFAULT gen_random_uuid(),
19     "title" VARCHAR(255) NOT NULL,
20     "description" TEXT,
21     "url" VARCHAR(500) NOT NULL,
22     "thumbnail" VARCHAR(500),
23     "views" INTEGER NOT NULL DEFAULT 0,
24     "created_at" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
25     "author_id" UUID NOT NULL,
26
27     CONSTRAINT "Video_pkey" PRIMARY KEY ("id")
28 );
29
30 -- CreateTable
31 CREATE TABLE "Comment" (
32     "id" UUID NOT NULL DEFAULT gen_random_uuid(),
33     "text" TEXT NOT NULL,
34     "created_at" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
35     "user_id" UUID NOT NULL,
36     "video_id" UUID NOT NULL,
37
38     CONSTRAINT "Comment_pkey" PRIMARY KEY ("id")
39 );
40
41 -- CreateTable
42 CREATE TABLE "Like" (
43     "id" UUID NOT NULL DEFAULT gen_random_uuid(),
44     "created_at" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
45     "user_id" UUID NOT NULL,
46     "video_id" UUID NOT NULL,
47

```

Міграція 1 — init: індекси та зовнішні ключі (частина 2)

```

MyWork > Lab_6 > prisma > migrations > 20260916000000_init > migration.sql
51 -- CreateTable
52 CREATE TABLE "Subscription" (
53   "id" UUID NOT NULL DEFAULT gen_random_uuid(),
54   "created_at" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
55   "subscriber_id" UUID NOT NULL,
56   "subscribed_to_id" UUID NOT NULL,
57
58   CONSTRAINT "Subscription_pkey" PRIMARY KEY ("id")
59 );
60
61 -- CreateIndex
62 CREATE UNIQUE INDEX "User_email_key" ON "User"("email");
63
64 -- CreateIndex
65 CREATE UNIQUE INDEX "User_username_key" ON "User"("username");
66
67 -- CreateIndex
68 CREATE UNIQUE INDEX "User_google_id_key" ON "User"("google_id");
69
70 -- CreateIndex
71 CREATE UNIQUE INDEX "Like_user_id_video_id_key" ON "Like"("user_id", "video_id");
72
73 -- CreateIndex
74 CREATE UNIQUE INDEX "Subscription_subscriber_id_subscribed_to_id_key" ON "Subscription"("subscriber_id", "subscribed_to_id");
75
76 -- AddForeignKey
77 ALTER TABLE "Video" ADD CONSTRAINT "Video_author_id_fkey" FOREIGN KEY ("author_id") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE CASCADE;
78
79 -- AddForeignKey
80 ALTER TABLE "Comment" ADD CONSTRAINT "Comment_user_id_fkey" FOREIGN KEY ("user_id") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE CASCADE;
81
82 -- AddForeignKey
83 ALTER TABLE "Comment" ADD CONSTRAINT "Comment_video_id_fkey" FOREIGN KEY ("video_id") REFERENCES "Video"("id") ON DELETE CASCADE ON UPDATE CASCADE;
84
85 -- AddForeignKey
86 ALTER TABLE "Like" ADD CONSTRAINT "Like_user_id_fkey" FOREIGN KEY ("user_id") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE CASCADE;
87
88 -- AddForeignKey
89 ALTER TABLE "Like" ADD CONSTRAINT "Like_video_id_fkey" FOREIGN KEY ("video_id") REFERENCES "Video"("id") ON DELETE CASCADE ON UPDATE CASCADE;
90
91 -- AddForeignKey
92 ALTER TABLE "Subscription" ADD CONSTRAINT "Subscription_subscriber_id_fkey" FOREIGN KEY ("subscriber_id") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE CASCADE;
93
94 -- AddForeignKey
95 ALTER TABLE "Subscription" ADD CONSTRAINT "Subscription_subscribed_to_id_fkey" FOREIGN KEY ("subscribed_to_id") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE CASCADE;
96

```

Міграція 2 — add-video-duration (ALTER TABLE)

```

VSC projects > DB_labs > MyWork > Lab_6 > prisma > migrations > 20260916000001_add_video_duration > migration.sql
1 -- AlterTable: додати поле duration до Video
2 ALTER TABLE "Video" ADD COLUMN "duration" INTEGER;
3

```

Міграція 3 — add-video-category (нова таблиця + FK)

```
VSC projects > DB_labs > MyWork > Lab_6 > prisma > migrations > 20260916000002_add_video_category > migration.sql
1  -- CreateTable: Category
2  CREATE TABLE "Category" (
3    "id" SERIAL NOT NULL,
4    "name" VARCHAR(100) NOT NULL,
5
6    CONSTRAINT "Category_pkey" PRIMARY KEY ("id")
7  );
8
9  -- CreateIndex
10 CREATE UNIQUE INDEX "Category_name_key" ON "Category"("name");
11
12 -- AlterTable: додати categoryId до Video
13 ALTER TABLE "Video" ADD COLUMN "category_id" INTEGER;
14
15 -- AddForeignKey
16 ALTER TABLE "Video" ADD CONSTRAINT "Video_category_id_fkey" FOREIGN KEY ("category_id") REFERENCES "Category" ("id");
17
```

Prisma Studio — таблиця User з даними

Search models	Like X	Subscription X	User X	Comment X	Category X	Close all
All Models	Filters	None	Fields	All	Showing	5 of 5
Category	0					
Comment	5					
Like	5					
Subscription	5					
User	5					
Video	5					

id A	email A	username A	password A?	googleid A?	description A?	avatar A?	banner A?
a1b2c3d4-0001-0000-0000...	denys@example.com	denys_dev	hashed_pass_1	null	Full-stack developer. U...	https://cdn.videohub.co...	https://cdn.videohub.co...
a1b2c3d4-0002-0000-0000...	maria@example.com	maria_gaming	hashed_pass_2	null	Gaming streamer. Love R...	https://cdn.videohub.co...	null
a1b2c3d4-0003-0000-0000...	oleg@example.com	oleg_cook	null	google_auth_id_3	Chef sharing recipes an...	https://cdn.videohub.co...	https://cdn.videohub.co...
a1b2c3d4-0004-0000-0000...	anna@example.com	anna_travel	hashed_pass_4	null	Traveling the world, on...	null	null
a1b2c3d4-0005-0000-0000...	max@example.com	max_music	null	google_auth_id_5	Musician. Covers, origi...	https://cdn.videohub.co...	https://cdn.videohub.co...

Prisma Studio — таблиця Like з даними

Search models	Like X	Subscription X	User X	Comment X	Category X	Close all
All Models	Filters	None	Fields	All	Showing	5 of 5
Category	0					
Comment	5					
Like	5					
Subscription	5					
User	5					
Video	5					

id A	createdAt	userId A	videoId A	user ()	video ()
d4e5f6a7-0001-0000-0000...	2026-09-01T23:28:07.333Z	a1b2c3d4-0002-0000-0000...	b2c3d4e5-0001-0000-0000...	User	Video
d4e5f6a7-0002-0000-0000...	2026-09-01T23:28:07.333Z	a1b2c3d4-0001-0000-0000...	b2c3d4e5-0002-0000-0000...	User	Video
d4e5f6a7-0003-0000-0000...	2026-09-01T23:28:07.333Z	a1b2c3d4-0004-0000-0000...	b2c3d4e5-0003-0000-0000...	User	Video
d4e5f6a7-0004-0000-0000...	2026-09-01T23:28:07.333Z	a1b2c3d4-0005-0000-0000...	b2c3d4e5-0004-0000-0000...	User	Video
d4e5f6a7-0005-0000-0000...	2026-09-01T23:28:07.333Z	a1b2c3d4-0003-0000-0000...	b2c3d4e5-0005-0000-0000...	User	Video

Походження бази даних

База даних **VideoHub** була взята з реального проєкту — відеохостинг-платформи на стеку Next.js + Prisma + PostgreSQL. Проєкт знаходиться в розробці, і схема БД продовжує дороблюватись в процесі проходження лабораторних робіт.

Як БД використовувалась у лабораторних:

- **Lab 1** — концептуальна модель (ER-діаграма) на основі схеми VideoHub
- **Lab 2** — DDL: фізична схема в PostgreSQL (CREATE TABLE, INSERT) на основі Prisma-схеми проєкту
- **Lab 3** — OLTP: CRUD-операції (SELECT, INSERT, UPDATE, DELETE) на тестових даних
- **Lab 4** — OLAP: агрегація, GROUP BY, HAVING, JOIN на тестових даних

- **Lab 5** — нормалізація: перевірка схеми на 1NF, 2NF, 3NF, виправлення через ALTER TABLE
- **Lab 6** — Prisma ORM: опис схеми в `schema.prisma`, міграції, Prisma Studio

Тестові дані:

Для тестування БД використовувались тестові дані з файлу `prisma/seed.sql` — 5 користувачів, 5 відео, 5 коментарів, 5 лайків, 5 підписок. Ці дані дозволяють перевірити всі операції (CRUD, агрегацію, JOIN, нормалізацію) на реалістичних прикладах.

Таблиця	Кількість	Приклади
User	5	denys_dev, maria_gaming, oleg_cook, anna_travel, max_music
Video	5	PostgreSQL Tutorial, Elden Ring, Pasta Carbonara, Tokyo, Wonderwall
Comment	5	коментарі до відео різних авторів
Like	5	лайки між користувачами
Subscription	5	підписки між користувачами

Чому БД з проєкту, а не з нуля:

Використання реальної схеми проєкту дає кілька переваг:

- Схема вже нормалізована і відповідає реальним вимогам
- Тестові дані реалістичні (не абстрактні Item/Creature)
- Лабораторні роботи логічно пов'язані — кожна наступна будується на попередній
- Prisma-схема з проєкту використовується як основа для Lab 6

Будь які припущення чи обмеження

- База даних VideoHub була взята з реального проєкту (відеохостинг-платформа на Next.js + Prisma + PostgreSQL). Схема дороблювалась в процесі лабораторних робіт. Тестові дані для перевірки — у файлі `prisma/seed.sql`.
- `DATABASE_URL` містить облікові дані PostgreSQL. В реальному проєкті `.env` не комітиться в Git (в `.gitignore`). Для лабораторної створено `.env.example` з прикладом.
- `@map` та `@@map` використовуються для мапінгу camelCase (Prisma/TypeScript) → snake_case (PostgreSQL). Наприклад, `authorId` в коді → `author_id` в БД.
- `onDelete: Cascade` на `@relation` — при видаленні батьківського запису автоматично видаляються дочірні (як ON DELETE CASCADE в SQL з Lab 2).
- `ON DELETE SET NULL` на `Video→Category` — при видаленні категорії відео не видаляється, `category_id` стає NULL. Це безпечніше за CASCADE для категорій.
- `@default(autoincrement())` для `Category.id` — SERIAL в PostgreSQL, автоінкремент. На відміну від UUID, числовий id зручніший для категорій (невеликий справочник).
- Prisma migrate dev автоматично: 1) порівнює `schema.prisma` з БД, 2) генерує SQL-міграцію, 3) застосовує до БД, 4) генерує клієнт. В продакшені використовується `prisma migrate deploy` (без генерації).

- Міграції зберігаються в `prisma/migrations/` — це історія змін схеми. Кожна міграція має timestamp + name + migration.sql. Дозволяє відкотити зміни або відтворити схему на новій БД.